

Supervised Learning - Part 2

COMPLETE TUTORIAL: Decision Trees, KNN, SVM, Neural Networks & Autoencoders

GARP RAI Exam Preparation

What You Will Learn in This Tutorial:

Topic	Key Concepts Covered
1. Decision Trees	CART, RSS, Entropy, Gini, Pruning, Ensemble Methods
2. K-Nearest Neighbors	Lazy Learner, Distance Metrics, K Selection, Curse of Dimensionality
3. Support Vector Machines	Margin, Support Vectors, Kernel Trick, Soft Margin
4. Neural Networks	MLP, Activation Functions, Backpropagation, Overfitting
5. Autoencoders	Encoder-Decoder, Dimensionality Reduction, Anomaly Detection

Tutorial Format:

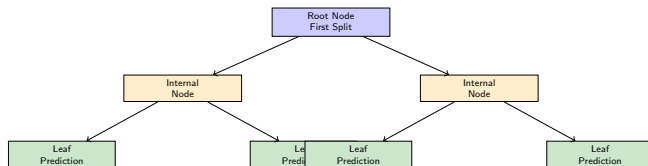
- Each topic presented with **theory**, **mathematics**, **examples**, and **exam traps**
- Step-by-step explanations with visual descriptions
- Practical tips for risk management applications

Key to Success: Understand the "**why**" behind each algorithm, not just the "**how**".

What is a Decision Tree?

- A supervised machine learning technique
- Uses a tree-like structure for predictions
- Sequentially splits data based on feature values
- Can handle both classification and regression (CART)

Anatomy of a Decision Tree:



Key Insight: Decision trees are "white-box" models - highly interpretable!

DECISION TREES - Why "White-Box"?

White-Box vs Black-Box:

White-Box Models	Black-Box Models
Decision Trees	Neural Networks
Easy to understand	Difficult to interpret
Traceable decisions	Hidden decision process
Visual representation	Complex mathematics
Regulatory friendly	Regulatory challenges

Example - Credit Decision:

- Tree: "Is credit score > 700 ?" $\rightarrow$ "Is income $> \$80,000$?" $\rightarrow$ Default Probability = 2%
- Easy to explain to regulators!

Exam Trap: Interpretability vs Accuracy trade-off - ensembles improve accuracy but reduce interpretability.

Regression Trees - For Continuous Target Variables

Example: Predicting house price per square foot using:

- Feature 1: Age of house
- Feature 2: Distance to metro station

How They Work:

- 1 Partition feature space into **non-overlapping regions**
- 2 Prediction = **average** of training observations in that region
- 3 Split criterion: **Minimize RSS**

Key Formula - Residual Sum of Squares (RSS):

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \bar{y}_{R_j})^2$$

where $\bar{y}_{R_j}$ = mean of observations in region j

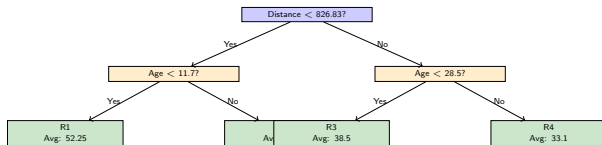
Exam Trap: The value that minimizes RSS in a region is the **MEAN**, not the median or mode!

DECISION TREES - Regression Trees Example

House Price Prediction Example:

Step 1: Find first split - Distance to metro < 826.83 feet?

Step 2: Split left branch - Age < 11.7 years?



Interpretation:

- **R1:** Close to metro, new houses → Highest price (52.25)
- **R4:** Far from metro, older houses → Lowest price (33.1)

How Trees Are Built - The Algorithm:

Why Not Try All Possibilities?

- Number of possible tree structures is **astronomical**
- Computationally **infeasible** for any real dataset
- Need a **pragmatic approach**

Recursive Binary Splitting:

- 1 Start with **all data** in one region (root)
- 2 Search every feature and every split point
- 3 Find split that **maximizes RSS reduction** (regression) or **maximizes purity** (classification)
- 4 Split data into two regions
- 5 **Repeat** independently on each region
- 6 Stop when **stopping criterion** is met

Stopping Criteria:

- Minimum observations per leaf
- Maximum tree depth
- Maximum number of leaves
- Minimum information gain

Exam Trap: This is a **"greedy"** algorithm - it makes locally optimal choices, not globally optimal ones!

Classification Trees - For Categorical Target Variables

Example: Predicting whether a firm will pay dividends

Key Difference from Regression Trees:

- Goal: Assign observations to a class
- Splitting criterion: Maximize node purity
- Measures of impurity: Entropy or Gini Coefficient

Entropy Formula:

$$Entropy = - \sum_{j=1}^J p_j \log_2(p_j)$$

Gini Coefficient Formula:

$$Gini = 1 - \sum_{j=1}^J p_j^2$$

Interpretation:

- Lower values = more pure nodes
- Perfect purity = all observations in one class $\rightarrow$ Entropy = 0, Gini = 0
- Maximum impurity = equal class distribution $\rightarrow$ Entropy = 1 (binary), Gini = 0.5 (binary)

Exam Trap: Entropy and Gini usually lead to similar trees - don't overthink the choice!

DECISION TREES - Entropy Calculation Example

Entropy - Worked Example:

Scenario: Node with 100 observations: 70 Non-Default, 30 Default

Step 1: Calculate probabilities:

- $p_{\text{Non-Default}} = 70/100 = 0.7$
- $p_{\text{Default}} = 30/100 = 0.3$

Step 2: Apply entropy formula:

$$\text{Entropy} = -[0.7 \log_2(0.7) + 0.3 \log_2(0.3)]$$

Step 3: Calculate:

- $\log_2(0.7) \approx -0.5146$
- $\log_2(0.3) \approx -1.7370$
- $\text{Entropy} = -[0.7 \times (-0.5146) + 0.3 \times (-1.7370)]$
- $\text{Entropy} = -[-0.3602 - 0.5211] = 0.8813$

Interpretation:

- Entropy ranges from 0 (pure) to 1 (maximum impurity for binary)
- 0.8813 indicates moderate impurity
- Not a perfect split, but better than 50-50

DECISION TREES - Gini Calculation Example

Gini Coefficient - Worked Example:

Same Scenario: Node with 100 observations: 70 Non-Default, 30 Default

Step 1: Calculate probabilities:

- $p_{\text{Non-Default}} = 0.7$
- $p_{\text{Default}} = 0.3$

Step 2: Apply Gini formula:

$$Gini = 1 - (0.7^2 + 0.3^2)$$

Step 3: Calculate:

- $0.7^2 = 0.49$
- $0.3^2 = 0.09$
- $\text{Sum} = 0.49 + 0.09 = 0.58$
- $Gini = 1 - 0.58 = 0.42$

Comparison Table:

Node Distribution	Entropy	Gini
100% Pure (70-0 or 0-70)	0	0
80-20 split	0.722	0.32
70-30 split	0.881	0.42
50-50 split	1.0	0.5

Exam Trap: Both measures **decrease** as **purity increases** - they give similar results!

Information Gain - Selecting the Best Split

What is Information Gain?

- The **reduction in impurity** achieved by a split
- Formula: $IG = \text{Parent Impurity} - \text{Weighted Child Impurities}$

Dividend Payment Example:

Step 1: Baseline Gini = 0.480 (no split)

Step 2: Split on "Large.Cap":

- Large Cap (13 firms): 11 paid, 2 not $\rightarrow$ Gini = 0.260
- Non-Large Cap (7 firms): 1 paid, 6 not $\rightarrow$ Gini = 0.245
- Weighted Gini = $(13/20 \times 0.260) + (7/20 \times 0.245) = 0.255$

Step 3: Calculate Information Gain:

$$IG = 0.480 - 0.255 = 0.225$$

Step 4: Repeat for all features $\rightarrow$ Choose feature with **highest IG**

Exam Trap: Information gain is **ALWAYS ≥ 0** ! A good split reduces impurity.

Handling Continuous Features in Decision Trees

The Challenge:

- Continuous features can take **any value** (e.g., Retail_Investor = 0-100%)
- Need to find the **optimal threshold** for splitting

Algorithm:

- 1 Sort the continuous feature values
- 2 Consider each distinct value as a **potential threshold**
- 3 Calculate information gain for **each** threshold
- 4 Select the threshold that **maximizes** information gain

Example - Retail_Investor:

- Values: 20%, 30%, 40%, 50%, 60%, 70%, 80%
- Test threshold: 35% → Left: $\leq 35\%$, Right: $> 35\%$
- Test threshold: 45% → Left: $\leq 45\%$, Right: $> 45\%$
- Choose the threshold with **highest information gain**

Key Insight:

- This is an **automated** process in machine learning packages
- The optimal threshold depends on the **node** (not fixed globally!)

Pruning - Reducing Tree Complexity

Why Prune?

- 1 Prevent overfitting
- 2 Improve interpretability
- 3 Remove irrelevant features

Two Types of Pruning:

Pre-Pruning (Online)	Post-Pruning (Offline)
During tree growth	After tree is fully grown
Stopping criteria	Remove weak links
Minimum leaf size	Reduced error pruning
Maximum depth	Cost-complexity pruning
Minimum information gain	Bottom-up approach
Computationally efficient	More computationally intensive

Reduced Error Pruning:

- 1 Start at **leaves** (bottom-up)
- 2 Replace a node with its **most popular class**
- 3 Accept if **validation performance doesn't degrade**

Exam Trap: Post-pruning often finds **better trees** but is more computationally expensive!

Cost-Complexity Pruning - Mathematical Formulation

Objective Function:

$$\text{Minimize } RSS + \alpha \times (\text{Number of Leaves})$$

Components:

- RSS : Residual Sum of Squares (fit to training data)
- α : **Complexity parameter** (tuning parameter)
- Number of Leaves: Model complexity

Effect of α :

- $\alpha = 0$: No penalty $\rightarrow$ **Largest tree** (overfit)
- α very large: Heavy penalty $\rightarrow$ **Smallest tree** (underfit)
- Optimal α : **Best balance** of fit and complexity

Selecting α :

- 1 Define a grid of α values
- 2 For each α , build and prune a tree
- 3 Use **cross-validation** to select best α
- 4 Choose α that minimizes validation error

Exam Trap: Cross-validation is the **standard method** for selecting α - not arbitrary choice!

Ensemble Method 1: Bagging (Bootstrap Aggregation)

Core Idea:

- Create **multiple** decision trees
- Each tree trained on a **different bootstrap sample**
- Aggregate predictions

Algorithm:

- 1 Sample with replacement (**bootstrapping**) from training data
- 2 Train a decision tree on each sample
- 3 Repeat many times (e.g., 500 trees)
- 4 Aggregation: **Average** (regression) or **Majority Vote** (classification)

Example:

- Training set: 100,000 observations
- Each bootstrap sample: 10,000 observations (with replacement)
- About **63.2%** of observations appear in each sample
- **36.8%** are **Out-of-Bag (OOB)** - used for validation

Why It Works:

- Reduces **variance** by averaging models
- If models are uncorrelated: $Var(\bar{Y}) = Var(Y)/M$

Ensemble Method 2: Random Forests

Improvement Over Bagging:

- Same as bagging **PLUS** feature randomization
- At each split, consider only a **random subset** of features
- Typical: $\sqrt{p}$ features where p = total features

Why Feature Randomization?

- Bagging trees can be **highly correlated**
- If a feature is very strong, it appears in **all** trees
- Correlation $\rightarrow$ less benefit from averaging
- Random forests $\rightarrow$ **de-correlated** trees

Comparison:

Feature	Bagging	Random Forest
Feature subset	All features	$\sqrt{p}$ features
Tree correlation	High	Low
Variance reduction	Moderate	High

Exam Trap: Random forests sacrifice individual tree optimality for **ensemble diversity**!

Ensemble Method 3: Boosting

Core Idea:

- Build trees **sequentially**
- Each new tree corrects errors of previous trees

Gradient Boosting:

- 1 Fit a simple tree to the data
- 2 Calculate **residuals** (errors)
- 3 Fit next tree to the **residuals**
- 4 Combine predictions
- 5 Repeat, each tree fits residuals of the combined model

AdaBoost:

- 1 Start with **equal weights** on all observations
- 2 Train a tree
- 3 Increase weights of **misclassified** observations
- 4 Train next tree with updated weights
- 5 Final model: **weighted sum** of trees

Exam Trap: Boosting is **sequential** (not parallel) and can **overfit** if not carefully tuned!

DECISION TREES - Boosting vs Bagging Summary

Boosting vs Bagging - Key Differences:

Aspect	Bagging/RF	Boosting
Training	Parallel	Sequential
Focus	Reduce variance	Reduce bias
Overfitting Risk	Low	High
Interpretability	Low	Very Low
Speed	Fast	Slow

When to Use Each:

● Bagging/Random Forests:

- When you have many features
- When you want robust performance
- When you want to avoid overfitting

● Boosting:

- When you need maximum accuracy
- When you have enough data to validate carefully
- When you are willing to do extensive hyperparameter tuning

Exam Trap: Boosting is **more powerful** but **more dangerous** (overfitting)!

Decision Trees - Practical Tips for Risk Management

Advantages:

- 1 Highly interpretable - easy to explain to regulators
- 2 Handles both continuous and categorical features
- 3 No feature scaling required
- 4 Handles missing values (in some implementations)

Disadvantages:

- 1 Prone to overfitting (single tree)
- 2 Lower accuracy than more complex models
- 3 Unstable - small data changes can create different trees
- 4 Biased toward features with many levels

Risk Management Applications:

- Credit scoring (default prediction)
- Fraud detection
- Operational risk classification
- Regulatory capital calculations

Exam Trap: In risk management, [interpretability](#) is often as important as [accuracy](#)!

K-Nearest Neighbors (KNN) - Overview

What is KNN?

- **Supervised** learning algorithm
- Used for **classification** and **regression**
- **Non-parametric** - no underlying distribution assumed
- **Instance-based** - memorizes training data

Why "Lazy Learner"?

- No explicit training phase
- Just **stores** the training data
- All "work" happens at **prediction time**

Core Idea:

- "Tell me who your neighbors are, and I'll tell you who you are"
- For a new instance, find the **K closest** training instances
- Classification: **Majority vote** of neighbors
- Regression: **Average** of neighbors' values

Key Hyperparameters:

- 1 **K**: Number of neighbors to consider
- 2 **Distance Metric**: How to measure closeness
- 3 **Weighting**: Equal or distance-weighted

KNN - Distance Metrics

Distance Metrics in KNN

Euclidean Distance (L2 Norm):

$$d_E(x, x') = \sqrt{\sum_{j=1}^m (x_j - x'_j)^2}$$

- Most common choice
- Straight-line distance
- Sensitive to scale

Manhattan Distance (L1 Norm):

$$d_M(x, x') = \sum_{j=1}^m |x_j - x'_j|$$

- Sum of absolute differences
- More robust to outliers
- Less sensitive to scale

Other Metrics:

- Cosine similarity (text data)
- Minkowski distance (generalization of L1 and L2)
- Hamming distance (binary/categorical data)

Which to Choose?

- **Euclidean**: Continuous, well-scaled features
- **Manhattan**: High-dimensional data
- **Hamming**: Binary/categorical features

Exam Trap: Choice of distance metric **significantly affects** KNN performance!

Why Feature Scaling is CRITICAL for KNN

The Problem:

- KNN relies on distance calculations
- Features with **larger scales dominate** the distance
- Smaller-scale features become **irrelevant**

Example - Without Scaling:

Feature	Range	Scale
Income	\$20,000 - \$500,000	Large
Age	18 - 80	Small

- Income dominates distance calculation
- Age has almost no effect
- **Model effectively ignores age!**

Solution - Scaling Methods:

- 1 **Standardization:** $x' = \frac{x - \mu}{\sigma}$ (mean 0, std 1)
- 2 **Min-Max Normalization:** $x' = \frac{x - \min}{\max - \min}$ (range 0 to 1)

Exam Trap: ALWAYS scale features before applying KNN - it's not optional!

How to Choose K in KNN

Effect of K:

Small K	Large K
More complex model	Simpler model
Overfitting risk	Underfitting risk
Sensitive to noise	Smoother boundaries
High variance	High bias

Heuristics:

- 1 $K = \sqrt{N}$ where N = training set size
- 2 $K = \log(N)$ for very large datasets
- 3 Use odd K to avoid ties in classification

Cross-Validation Method:

- 1 Try a range of K values (e.g., 1, 3, 5, 7, ..., 50)
- 2 Use cross-validation to estimate test error
- 3 Select K with **lowest validation error**

Example:

- $N = 10,000 \rightarrow K = \sqrt{10,000} = 100$
- But cross-validation might suggest $K = 85$ or $K = 120$
- **Always validate!**

KNN - Curse of Dimensionality

Curse of Dimensionality in KNN

The Problem:

- As the number of features (m) increases
- Distances become **less meaningful**
- All points become approximately **equidistant**

Mathematical Insight:

- In high dimensions, the **ratio** of distances between nearest and farthest points approaches 1
- "Nearest neighbor" loses its meaning

Example:

- 1D space: Points clearly have nearest neighbors
- 2D space: Some points are clearly closer
- 100D space: All points are about the same distance from each other

Mitigation Strategies:

- 1 **Feature Selection**: Choose only relevant features
- 2 **Dimensionality Reduction**: PCA, Autoencoders
- 3 **Specialized Distance Metrics**: For high-dimensional data
- 4 **More Data**: Larger training sets help

Exam Trap: The curse of dimensionality makes KNN **less effective** with many features!

KNN - Advantages and Disadvantages

KNN - Summary of Pros and Cons

Advantages:

- 1 Simple and intuitive
- 2 No training phase - fast to "train"
- 3 No assumptions about data distribution
- 4 Can handle multi-class problems
- 5 Can handle regression and classification
- 6 Natural for multi-class problems

Disadvantages:

- 1 Computationally expensive for predictions ($O(N \times m)$)
- 2 Sensitive to feature scaling
- 3 Curse of dimensionality
- 4 Sensitive to noise and outliers
- 5 Cannot handle missing values without imputation
- 6 Not interpretable (no model to explain)

When to Use KNN:

- Small to medium datasets
- Well-scaled, low-dimensional features
- When interpretability is not critical
- When you need a simple baseline model

Exam Trap: KNN is great for small datasets but struggles with large ones!

KNN - Worked Example

KNN Classification - Step-by-Step Example

Data: Borrower Default Prediction

ID	Balance	Income	Default
1	-0.55	-0.75	0 (No)
2	-0.23	0.19	0
3	-0.76	0.53	0
4	-0.08	-0.73	0
5	-0.72	1.32	0
6	-0.79	2.00	0
7	-0.80	-0.34	0
8	0.46	-0.71	1 (Yes)
9	1.95	-0.96	1
10	1.51	-0.54	1

New Borrower:

- Balance = 0.90
- Income = -0.61

Step 1: Choose $K = 4$

Step 2: Calculate distances (Euclidean)

- To ID 8: $\sqrt{(0.46 - 0.90)^2 + (-0.71 + 0.61)^2} = 0.451$
- To ID 10: $\sqrt{(1.51 - 0.90)^2 + (-0.54 + 0.61)^2} = 0.611$
- To ID 4: $\sqrt{(-0.08 - 0.90)^2 + (-0.73 + 0.61)^2} = 0.994$
- To ID 9: $\sqrt{(1.95 - 0.90)^2 + (-0.96 + 0.61)^2} = 1.105$

Step 3: 4 nearest: 8, 10, 4, 9 (3 Default, 1 Non-Default)

Step 4: Majority vote → **Default!**

Support Vector Machines (SVM) - Overview

What is SVM?

- Supervised learning algorithm
- Primarily for classification
- Finds the optimal separating hyperplane
- Can handle linear and non-linear boundaries

Core Concepts:

- 1 Hyperplane: Decision boundary
- 2 Margin: Distance between boundary and closest points
- 3 Support Vectors: Points that define the margin

Key Idea:

- Find the hyperplane that maximizes the margin
- Wider margin → better generalization

SVM Optimization:

$$\min_w \frac{1}{2} |w|^2$$

subject to: $y_j(w_0 + w^T x_j) \geq 1$ for all j

Exam Trap: Maximizing margin $\Leftrightarrow$ Minimizing $|w|$!

SVM - The Mathematics Behind It

For 2D Case:

$$D(x) = w_0 + w_1x_1 + w_2x_2$$

Margin Constraints:

- Upper margin: $w_0 + w_1x_1^{(u+)} + w_2x_2^{(u+)} = 1$
- Lower margin: $w_0 + w_1x_1^{(l-)} + w_2x_2^{(l-)} = -1$

Margin Width:

$$\text{Margin Width} = \frac{2}{|w|}, \quad |w| = \sqrt{w_1^2 + w_2^2}$$

Classification Rule:

- $D(x) > 0 \rightarrow \text{Class } +1$
- $D(x) < 0 \rightarrow \text{Class } -1$

Example - Car Loan:

- $w_0 = -12.24, w_1 = 0.90, w_2 = 1.26$
- Boundary: $-12.24 + 0.90(\text{Income}) + 1.26(\text{Savings}) = 0$
- New applicant: Income=5.7, Savings=3.5
- $D(x) = -12.24 + 0.90(5.7) + 1.26(3.5) = -2.7$
- **Loan Not Granted!**

SVM - Support Vectors

Support Vectors - The Critical Points

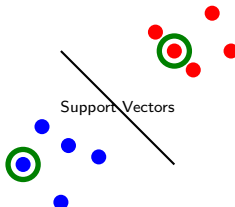
What are Support Vectors?

- Data points that lie **exactly on the margin**
- The **closest** points to the hyperplane
- They **"support"** the boundary

Why Only Support Vectors Matter:

- If a non-support vector moves, **boundary doesn't change**
- If a support vector moves, **boundary changes**
- SVM solution depends **only** on support vectors

Visual Representation:



Exam Trap: Only **support vectors** influence the boundary!

Soft Margin SVM - Handling Non-Separable Data

The Problem:

- Real data is often **not perfectly separable**
- Hard margin SVM **fails** on overlapping data

Soft Margin Solution:

- Allow some points to be **misclassified**
- Or allow points to fall **inside the margin**
- Introduce a **penalty** for misclassifications

Hinge Loss:

$$L(y, f(x)) = \max(0, 1 - yf(x))$$

SVM Objective with Soft Margin:

$$\min_w \frac{1}{N} \sum_{j=1}^N \max(0, 1 - y_j(w_0 + w^T x_j)) + \frac{\lambda}{2} |w|^2$$

Parameter λ (or C):

- λ large $\rightarrow$ **Wider margin**, more misclassifications
- λ small $\rightarrow$ **Narrower margin**, fewer misclassifications

Exam Trap: λ controls the **trade-off** between margin width and misclassification penalty!

The Kernel Trick - SVM's Secret Weapon

The Problem:

- Some data is **not linearly separable** in original space
- Example: Concentric circles - no straight line can separate them

The Solution:

- Project data to a **higher-dimensional space**
- It may become **linearly separable** there
- Example: 2D circles $\rightarrow$ 3D linear separator

The "Trick":

- SVM doesn't need the projected coordinates!
- Only needs the **kernel function**
- Kernel computes dot products in the high-dimensional space

Common Kernels:

- 1 **Linear:** $K(x, x') = x^T x'$
- 2 **Polynomial:** $K(x, x') = (x^T x' + 1)^d$
- 3 **RBF:** $K(x, x') = \exp(-\gamma \|x - x'\|^2)$

Exam Trap: The kernel trick enables **non-linear** classification without explicit high-dimensional computation!

RBF Kernel - The Most Popular Choice

Formula:

$$K(x, x') = \exp(-\gamma \|x - x'\|^2)$$

Parameter γ :

- Controls the **influence radius** of each training point
- Large $\gamma \rightarrow$ **Small influence radius**
 - Each point affects only nearby points
 - Complex, wiggly decision boundary
 - **Risk of overfitting**
- Small $\gamma \rightarrow$ **Large influence radius**
 - Each point affects many points
 - Smooth, simple decision boundary
 - **Risk of underfitting**

Intuition:

- γ controls the **smoothness** of the decision boundary
- Think of it as the "**radius of influence**"

Tuning γ :

- 1 Try multiple γ values (e.g., 0.001, 0.01, 0.1, 1, 10)
- 2 Use **cross-validation** to select best
- 3 Combine with C tuning

Exam Trap: γ is NOT the same as C - they control different aspects of the model!

SVM Hyperparameter Tuning - Grid Search

Why Tune?

- SVM performance is **highly sensitive** to parameters
- Poor choice → **Poor performance**

Key Parameters to Tune:

- 1 C (or λ): Soft margin penalty
- 2 γ (RBF): Kernel influence radius
- 3 d (Polynomial): Degree of polynomial

Grid Search Process:

- 1 Define grid: $C \in [0.1, 1, 10, 100]$, $\gamma \in [0.01, 0.1, 1, 10]$
- 2 For each combination:
 - Train SVM
 - Evaluate via cross-validation
- 3 Select combination with **lowest validation error**

Example Grid:

C	γ	Validation Accuracy
0.1	0.01	0.85
1	0.1	0.88
10	1	0.87
100	10	0.82

Exam Trap: Always tune hyperparameters; defaults are rarely optimal!

SVM - Advantages and Disadvantages

SVM - Summary of Pros and Cons

Advantages:

- 1 Effective in high-dimensional spaces
- 2 Memory efficient (uses support vectors only)
- 3 Versatile through kernel trick
- 4 Robust to overfitting (margin maximization)
- 5 Theoretically well-founded

Disadvantages:

- 1 Computationally expensive for large datasets ($O(N^2)$ - $O(N^3)$)
- 2 Does not directly provide probability estimates
- 3 Sensitive to hyperparameter tuning
- 4 Less interpretable with non-linear kernels
- 5 Performance degrades with noise/overlapping classes

When to Use SVM:

- Medium-sized datasets
- High-dimensional problems
- When accuracy is critical
- When you have time for careful tuning

Exam Trap: SVM is powerful but requires careful tuning and is not for massive datasets!

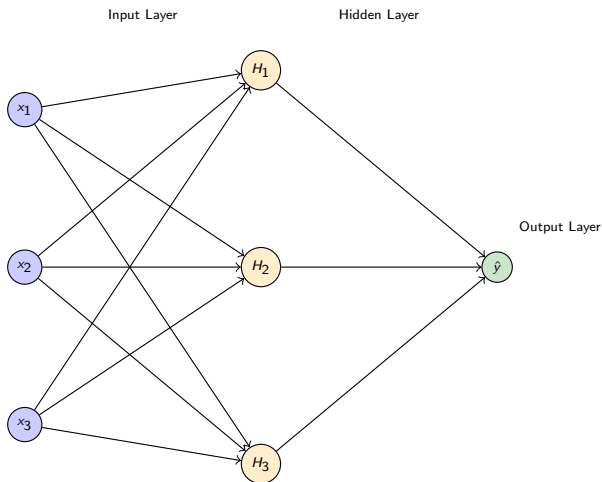
Neural Networks - Introduction

Neural Networks - The Building Blocks

What are Neural Networks?

- Inspired by the [human brain](#)
- Composed of [neurons](#) (nodes) organized in layers
- [Feedforward](#) networks (MLP) are most common

Basic Architecture:



Neuron Computation - Step by Step

Step 1: Weighted Sum

$$z = \sum_{i=1}^m w_i x_i + b$$

where:

- w_i = weights
- x_i = inputs
- b = bias term

Step 2: Activation Function

$$h = f(z)$$

where f is a non-linear function

Example - Hidden Neuron Calculation:

- Inputs: $x_1 = 0.6, x_2 = -0.4, x_3 = 0.3, x_4 = -0.2$
- Weights: $w_1 = 0.15, w_2 = -0.4, w_3 = -0.3, w_4 = 0.2$
- Bias: $b = 0.7$

$$z = 0.15(0.6) + (-0.4)(-0.4) + (-0.3)(0.3) + 0.2(-0.2) + 0.7 = 0.82$$

$$h = \text{ReLU}(0.82) = 0.82$$

Key Insight:

- Without non-linear activation, the network is just **linear regression**
- Activation functions enable **complex, non-linear** learning

Neural Networks - Activation Functions

Common Activation Functions

Sigmoid (Logistic):

$$f(z) = \frac{1}{1 + e^{-z}}$$

- Output: $(0, 1)$
- Use: Binary classification output layer
- **Vanishing gradient problem**

ReLU (Rectified Linear Unit):

$$f(z) = \max(0, z)$$

- Output: $[0, \infty)$
- Use: **Most common** for hidden layers
- **Efficient, helps with vanishing gradients**
- **Dying ReLU problem**

Leaky ReLU:

$$f(z) = \begin{cases} 0.01z & z < 0 \\ z & z \geq 0 \end{cases}$$

- Fixes dying ReLU
- Small negative slope for $z < 0$

Tanh:

$$f(z) = \frac{e^{2z} - 1}{e^{2z} + 1}$$

- Output: $(-1, 1)$
- Use: Recurrent neural networks
- **Vanishing gradient**

Softmax:

$$f(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Neural Networks - Backpropagation

How Neural Networks Learn - Backpropagation

The Training Process:

Step 1: Forward Pass

- 1 Input $\rightarrow$ Hidden Layer $\rightarrow$ Output
- 2 Calculate prediction $\hat{y}$

Step 2: Calculate Loss

$$L = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

or other loss function

Step 3: Backward Pass

- 1 Compute gradient of loss with respect to each weight
- 2 Use the [chain rule](#) from calculus
- 3 Propagate errors backward through the network

Step 4: Update Weights

$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w}$$

where η is the [learning rate](#)

Repeat:

- For many [epochs](#) (passes through data)
- Until convergence or early stopping

Exam Trap: Backpropagation is just [gradient descent](#) applied to neural networks!

Overfitting in Neural Networks

Why Neural Networks Overfit:

- Too many parameters (weights)
- Can memorize training data
- Learns noise and artifacts

Signs of Overfitting:

- 1 Training error ↓, Validation error ↑
- 2 Different predictions from different training runs
- 3 Extremely high training accuracy, poor test accuracy

Prevention Techniques:

Technique	How It Works
L2 Regularization	Penalty on large weights
Dropout	Randomly drop neurons during training
Early Stopping	Stop when validation error increases
More Data	More training examples
Data Augmentation	Create variations of training data
Smaller Network	Fewer parameters

L2 Regularization:

$$L_{new} = L_{old} + \lambda \sum w_i^2$$

Exam Trap: Multiple techniques together are more effective than any single one!

Dropout - Ensemble in a Single Network

What is Dropout?

- Randomly **drop** neurons during training
- Each neuron has probability p of being dropped
- Common $p = 0.5$ (50% dropout)

How It Works:

- 1 For each training iteration:
 - Randomly select neurons to **drop**
 - Dropped neurons output 0
 - Only remaining neurons participate
- 2 Different network architecture each iteration
- 3 At test time: **All** neurons used (scaled)

Why It Works:

- Prevents **co-adaptation** of neurons
- Forces network to learn **robust** features
- Acts as an **ensemble** of many networks

Example:

- Network with 100 neurons, dropout $p = 0.5$
- Each iteration uses **about 50** neurons
- Over training, **many** different sub-networks
- Final model averages all sub-networks

Exam Trap: Dropout is applied **only during training**, not at test time!

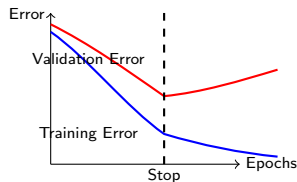
Neural Networks - Early Stopping

Early Stopping - The Simplest Regularization

The Concept:

- Monitor **validation error** during training
- Stop when validation error **starts to increase**
- Even if training error continues to decrease

Visual Representation:



Why It Works:

- Early training: Learning **general patterns**
- Late training: Learning **noise**
- Stop at optimal point of **generalization**

Practical Implementation:

- 1 Hold out 20-30% of data as validation set
- 2 Train network, monitor validation error
- 3 Stop when validation error hasn't improved for k epochs
- 4 Use model from **best** epoch

Convolutional Neural Networks (CNNs)

What CNNs Are Designed For:

- Data with **grid-like topology**
- **Images** (2D grid of pixels)
- Also: Text (1D grid), time series

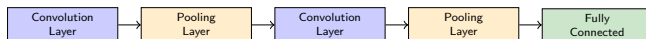
Key Innovation - Convolution:

- Uses **filters (kernels)** that slide across input
- Each filter detects a specific pattern
- Results in a **feature map**

Advantages of Convolution:

- 1 **Parameter Sharing**: Same filter applied everywhere
- 2 **Translation Invariance**: Pattern recognized regardless of location
- 3 **Sparse Connections**: Each output depends on small local region

CNN Architecture:



Exam Trap: CNNs are **not** just for images; they work on any grid-like data!

Neural Networks - CNN Components

CNN Components - Detailed Explanation

Convolutional Layer:

- Applies filters to input
- Each filter creates a **feature map**
- Number of filters $\rightarrow$ number of feature maps

Filter (Kernel) Example:

- 3x3 kernel slides over 4x4 image
- Each position produces one value
- Result: 2x2 feature map

Pooling Layer:

- Reduces **spatial dimensions**
- Common: Max pooling (take max value in window)
- Also: Average pooling
- Increases **translation invariance**

Pooling Example:

- 2x2 max pooling on 4x4 feature map
- Result: 2x2 feature map
- Takes maximum value from each 2x2 block

Fully Connected Layers:

- At the end of the network
- Use high-level features for classification
- Standard neural network layers

Exam Trap: Pooling **reduces** dimensions, convolution **preserves or changes** dimensions based on padding!

Neural Networks - RNNs

Recurrent Neural Networks (RNNs)

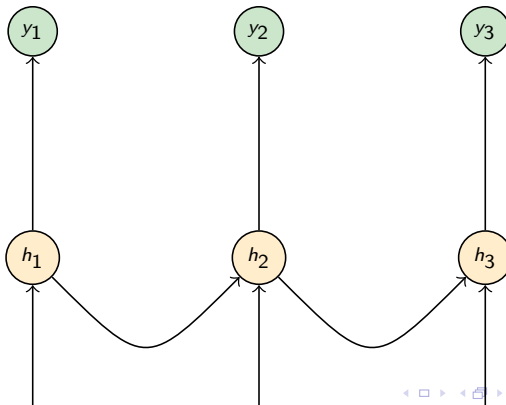
What Makes RNNs Special:

- Have **memory**
- Process **sequential** data
- Each output depends on previous inputs

Common Applications:

- 1 Time series forecasting
- 2 Natural language processing
- 3 Speech recognition
- 4 Text generation

RNN Structure:



Long Short-Term Memory (LSTM) Networks

The Problem with Basic RNNs:

- Cannot remember information for **long periods**
- **Vanishing gradients** in long sequences

How LSTMs Solve This:

- Add a **memory cell**
- Use **gates** to control information flow
- Can keep information for **long periods**

The Three Gates:

Gate	Function
Forget Gate	Decides what to discard from memory
Input Gate	Decides what new information to store
Output Gate	Decides what to output from memory

Memory Cell:

- Stores information over time
- Can hold information for **thousands of steps**
- Prevents vanishing gradients

Why LSTMs Are Popular:

- 1 **Handle long sequences well**
- 2 **State-of-the-art on many tasks**
- 3 **Used in transformers and language models**

Exam Trap: LSTMs are an **evolution** of RNNs, not a completely different architecture!

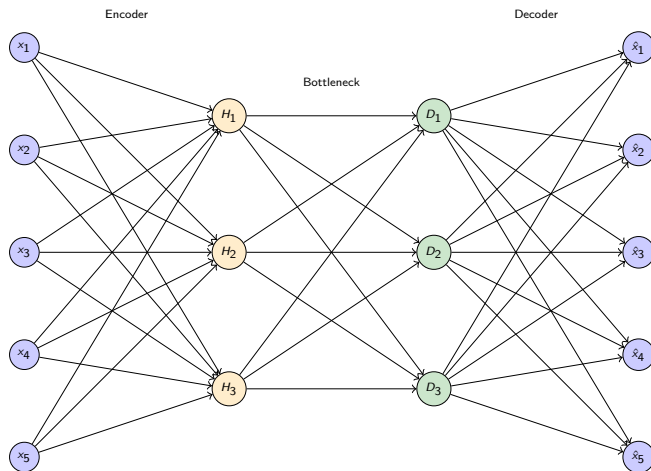
Autoencoders - Introduction

Autoencoders - Overview

What is an Autoencoder?

- **Unsupervised** neural network
- Learns to **compress** and **reconstruct** data
- Uses **no labels** - pure unsupervised learning

Architecture:



Key Components:

Autoencoders - How They Learn

How Autoencoders Learn - The Training Process

Training Objective:

- Minimize **reconstruction error**
- Input $\approx$ Output

Loss Function - Mean Squared Error:

$$L = \frac{1}{mN} \sum_{j=1}^m \sum_{i=1}^N (x_{ij} - \hat{x}_{ij})^2$$

The Bottleneck Effect:

- Hidden layer has **fewer neurons** than input
- Forces **compression**
- Network must learn **important features**

Example:

- Input: 100 features
- Bottleneck: 10 neurons
- Network learns to represent 100 features with just 10 numbers
- Reconstruction will lose some detail
- But **important patterns are preserved**

What the Network Learns:

- 1 **Most important features**
- 2 **Relationships between features**
- 3 **Dimensionality reduction**

Autoencoders - Linear vs Non-Linear

Linear Autoencoders vs Non-Linear Autoencoders

Linear Autoencoder:

- Uses **only linear** activation functions
- Equivalent to **PCA**
- Cannot capture non-linear relationships

Non-Linear Autoencoder:

- Uses **non-linear** activation functions (ReLU, sigmoid)
- Can capture **complex** relationships
- More powerful than PCA

Comparison Table:

Linear Autoencoder	Non-Linear Autoencoder
Equivalent to PCA	More powerful than PCA
No non-linearity	Non-linear activation functions
Unique solution	Many possible solutions
Fast training	Slower training
Less flexible	More flexible

When to Use Which:

- Linear: Simple data, need speed
- Non-linear: Complex data, need power

Exam Trap: A linear autoencoder with one hidden layer is **equivalent to PCA**!

Autoencoder Variants - Specialized Versions

Sparse Autoencoder:

- More hidden neurons than input
- But most weights are **near zero** (sparse)
- Uses L1 regularization
- Learns **sparse** representations

Denoising Autoencoder:

- Trained on **corrupted** inputs
- Reconstructs **clean** version
- Learns **robust** features
- Great for noise removal

Deep Autoencoder:

- Multiple hidden layers
- Symmetrical architecture
- Captures **hierarchical** features
- More powerful for complex data

Variational Autoencoder (VAE):

- Probabilistic version
- Learns **latent distribution**
- Can **generate** new data
- Used in generative AI

Exam Trap: Different variants serve **different purposes** - choose based on the task!

Autoencoders - Anomaly Detection

Autoencoders for Anomaly Detection - The Risk Management Application

The Process:

- 1 Train autoencoder on **normal data only**
- 2 The model learns to reconstruct normal patterns
- 3 For new data:
 - Calculate **reconstruction error**
 - Low error → **Normal**
 - High error → **Anomaly**

Why This Works:

- Anomalies are **different** from normal data
- Autoencoder trained only on normal data
- Cannot reconstruct anomalies well
- High reconstruction error indicates **anomaly**

Example - Fraud Detection:

- Train on normal transactions
- Fraudulent transaction → high reconstruction error
- Flag for investigation

Threshold Selection:

- Compute reconstruction errors on validation set
- Set threshold at, e.g., 95th percentile
- Error > threshold → **Anomaly**

Exam Trap: Autoencoders detect anomalies by **high reconstruction error** - not by classification!

Autoencoders - Advantages and Disadvantages

Autoencoders - Summary of Pros and Cons

Advantages:

- 1 Unsupervised - no labels needed
- 2 Can capture non-linear relationships
- 3 Great for dimensionality reduction
- 4 Excellent for anomaly detection
- 5 Can generate new data (VAEs)
- 6 Feature extraction for other models

Disadvantages:

- 1 Can be computationally expensive to train
- 2 Hyperparameter sensitive (architecture, learning rate)
- 3 Not interpretable - latent space is abstract
- 4 May not generalize to unseen data types
- 5 Requires careful tuning for good results

When to Use Autoencoders:

- Dimensionality reduction (when PCA isn't enough)
- Anomaly detection (fraud, operational risk)
- Feature learning (pre-training for other models)
- Data denoising (cleaning noisy data)

Exam Trap: Autoencoders are powerful but require careful design and tuning!

Autoencoders vs PCA - Detailed Comparison

Key Differences:

Autoencoder	PCA
Non-linear	Linear
Learns representation	Finds variance directions
Neural network	Matrix decomposition
Can be deep	Single transformation
Flexible architecture	Fixed mathematical solution
Sensitive to hyperparameters	No hyperparameters
Slower to train	Fast
Can handle complex data	Limited to linear relationships

When Autoencoders Win:

- Data has **non-linear** structure
- Need **more compact** representation
- Want to learn **features** for other tasks

When PCA Wins:

- Data is **linear**
- Need **fast** computation
- Need **interpretable** components
- Need **reproducible** results

Exam Trap: Autoencoders are **non-linear PCA** - choose based on data complexity!

Autoencoders - Treasury Yield Example

Autoencoders in Practice - Treasury Yield Data

Dataset:

- 8 Treasury yield series
- Daily changes in yields
- High-dimensional time series

Goal:

- Reduce dimensionality
- Capture yield curve dynamics
- Compare to PCA

Results:

Components	PCA MSE	Hidden Units	Autoencoder MSE
1	0.18208	1	0.18270
2	0.04965	2	0.05027
3	0.01708	3	0.01740
4	0.00848	4	0.00854
5	0.00528	5	0.00534
6	0.00295	6	0.00299
7	0.00107	7	0.00113
8	0.00000	8	0.00000

Interpretation:

- Autoencoder and PCA performed **similarly**
- Linear structure dominated yield curve
- Autoencoder didn't provide significant improvement
- **Not all problems benefit from non-linearity!**

TUTORIAL SUMMARY

Complete Tutorial Summary - Key Takeaways

Decision Trees:

- White-box models, highly interpretable
- Regression (RSS) vs Classification (Entropy/Gini)
- Pruning prevents overfitting
- Ensembles (Bagging, RF, Boosting) improve accuracy

K-Nearest Neighbors:

- Lazy learner, no training phase
- Distance-based, requires feature scaling
- K controls bias-variance trade-off
- Curse of dimensionality in high dimensions

Support Vector Machines:

- Maximum margin classifier
- Support vectors define the boundary
- Kernel trick enables non-linear boundaries
- Requires careful hyperparameter tuning

Neural Networks:

- Universal approximators
- Activation functions provide non-linearity
- Backpropagation for training
- Overfitting prevention: regularization, dropout, early stopping

Autoencoders:

- Unsupervised dimensionality reduction
- Encoder-Decoder architecture
- Can capture non-linear relationships
- Anomaly detection through reconstruction error

Exam Success Tip: Understand the "why" behind each algorithm, not just the "how"!

Top 10 Exam Traps to Remember:

- 1 **Trap 1:** Decision trees are **white-box** (interpretable), not black-box
- 2 **Trap 2:** Regression trees use **RSS**; classification trees use **Entropy/Gini**
- 3 **Trap 3:** KNN **requires feature scaling** - it's not optional!
- 4 **Trap 4:** Small K in KNN → **overfitting**; large K → **underfitting**
- 5 **Trap 5:** SVM maximizes **margin**; support vectors are the **closest points**
- 6 **Trap 6:** Kernel trick enables **non-linear** SVM without explicit high-dimensional computation
- 7 **Trap 7:** Neural networks need **non-linear** activation functions; linear = regression
- 8 **Trap 8:** Dropout is **only during training**, not at test time
- 9 **Trap 9:** Autoencoders are **unsupervised**; they reconstruct, not classify
- 10 **Trap 10:** Linear autoencoder with one hidden layer = **PCA**

Remember: The exam tests conceptual understanding, not just memorization!